ARCHITECTURE & MVP DESIGN

Project Fund Request and Liquidation Management System

How the MVP is built, why each choice was made, and how it scales from a one-host demo to a production deployment.

Document	Architecture & MVP design, version 1.0
Date	29 October 2026
Prepared by	Nandha Kumar
Based on	Problem statement (revised): six-stage workflow, four roles, FR-01 to FR-24
MVP time box	4 to 5 hours
Recommended stack	FastAPI + SQLAlchemy 2 · PostgreSQL · React (TypeScript) + Ant Design · Docker Compose

1. Summary
2. MVP scope
3. Roles and permissions
4. Technology choice
5. System architecture
6. Layered architecture
7. Workflow and state machine
8. How a request crosses the layers
9. Budget and money model
10. Data model
11. Concurrency and edge cases
12. Security
13. API and screens
14. Scalability roadmap
15. Testing strategy
16. Delivery plan
17. Design decisions log

1. Summary

The system controls project money from the moment an employee asks for it until Finance closes the books on it. A request passes through six stages (raise, approve, disburse, submit expenses, review expenses, verify and liquidate) and four roles (Requestor, Approver, Finance, Super Admin). Any rejection sends it back to a defined earlier stage.

Stack	Shape	Core rule	Scale path
FastAPI + SQLAlchemy, PostgreSQL, React + Ant Design, Docker Compose	One deployable app split into six business modules and five layers	Only the workflow engine changes a status; only the budget engine changes money	Stateless API, storage interface, outbox, ledger; no rewrite needed

What the architecture guarantees

- **No invalid status jump.** Every allowed transition is listed in one table (status, action, role, next status). Anything not in the table is refused with HTTP 409.
- **No over-commitment of a project budget.** Reservations lock the project row in PostgreSQL. A database CHECK constraint stops the total from passing the allocation even if the code has a bug.
- **No duplicate money movement.** Each status change only succeeds if the status is still the expected one. Disbursement takes an idempotency key, and the database allows one liquidation per request. A double click or a network retry can't pay or liquidate twice.
- **No hidden data.** Every read goes through a visibility filter on the backend, file downloads included. Requestors and approvers see only requests they're involved in.
- **Full history.** Every transition writes an audit row (user, time, old status, new status, reason, snapshot). Every money movement writes an append-only ledger row.

Why this stack, in one paragraph

The data is relational (projects, requests, items, documents, ledger) and the hard parts are transactional (locks, constraints, exact decimals). That makes **PostgreSQL** the fixed point of the design: it, not the web framework, enforces the guarantees above. On top of it we use **FastAPI with SQLAlchemy 2**. In a merit-only comparison (section 4), Django + DRF scored highest because it ships more built-in features. We still choose FastAPI for three reasons. It scores the same as Django and Spring Boot on the criterion that matters most here, transactional integrity. The team already builds and runs FastAPI with PostgreSQL in production, which removes delivery risk in a 4–5 hour window and means every line can be explained. And its explicit style (typed Pydantic contracts, visible transactions, a plain service layer) suits a workflow that reviewers will read line by line. Section 4.4 lists each Django built-in we give up and how the MVP covers it.

Where the PDF left decisions to us

The problem statement asks the implementer to define several money rules. Our decisions are listed in section 17: when budget is reserved and released, how a request returned after payout is handled, overspend, underspend, proof requirements and withdrawal. Each one has a short reason.

2. MVP scope

The MVP covers every item in the problem statement's "Expected deliverables" list. Optional enhancements are left out unless they cost almost nothing and make the demo clearer.

In the MVP

- Login (JWT) and role-based access on every endpoint
- User invitation by link, role assignment, activate / deactivate (Super Admin)
- Projects: create, edit, activate / deactivate, budget, requestor assignment
- Live budget figures: allocated, reserved, disbursed, liquidated, available
- All six stages with rejection and resubmission routing
- Approver reassignment while initial approval is pending
- Expense line items with document upload and item-level review
- Finance verification, liquidation, balance-return and top-up records
- Audit log and in-app notifications
- "My pending actions" dashboard, request list with filters and pagination
- Seed data, setup instructions, tests for workflow and money rules

Out of the MVP (and where it plugs in later)

- Email / Slack notifications: outbox table plus a Celery worker (section 14)
- Multi-level or configurable approval: the transition table moves into the database
- CSV / Excel export and PDF reports: read-only endpoints over the ledger
- Budget-exception workflow for cutting a budget below commitments
- Multi-tenancy (several organisations)
- Virus scanning of uploads

> **If time runs short, cut in this order:** audit-log screen (data is still recorded), notifications UI, advanced filters, project-member UI (seed members instead). The workflow, the money rules and the duplicate protection are never cut, because they are what is graded.

3. Roles and permissions

Every user has exactly one role. Super Admin can perform Finance's money actions only if the `can_finance_ops` flag is switched on for that user; it is off by default, which implements the PDF's "must be explicitly granted" rule.

Capability	Requestor	Approver	Finance	Super Admin
See projects	assigned only	of involved requests	**all**	**all**
Create / edit / activate / deactivate projects, set budgets	–	–	✓	✓
Assign requestors to projects	–	–	✓	✓
Raise, correct, resubmit, withdraw a fund request	**own**	–	–	–
Approve or reject the initial request	–	**assigned**	–	–
Reassign approver (only while PENDING_APPROVAL)	–	–	✓	✓
Disburse, or return a request to Stage 1	–	–	✓	if granted
Submit and correct expense items with proofs	**own**	–	–	–
Review expense items, give final approval	–	**assigned**	–	–

Capability	Requestor	Approver	Finance	Super Admin
Verify expense items, liquidate	–	–	✓	if granted
See requests	involved	involved	**all**	**all**
Invite users, assign roles, activate / deactivate accounts	–	–	–	✓
Audit log	own requests' history	involved requests' history	✓	✓

4. Technology choice

The options were first compared on merit alone, against what this application needs. Six criteria were weighted by how much they affect the graded outcome and the 4–5 hour time box, and each option was scored 1 (poor) to 5 (excellent). Team experience was then considered separately as a delivery-risk factor, and the final decision is explained openly in 4.3.

4.1 What the application demands

- **Transactional integrity:** two requests must not reserve the same budget; payouts and liquidations must not happen twice. This needs row locks, CHECK / UNIQUE constraints and exact decimal arithmetic.
- **Relational data:** project → request → expense items → documents, plus a ledger and an audit log that all reference each other.
- **A lot of standard business plumbing:** users, invitations, password hashing, role and object permissions, filtering, pagination, file uploads, migrations.
- **A short build window** with a clear path to scale afterwards.

4.2 Weighted comparison on merit only (backend + database)

Criterion (weight)	MERN (MongoDB)	NestJS + PostgreSQL	Spring Boot + PostgreSQL	FastAPI + PostgreSQL	Django DRF + PostgreSQL
Transactional integrity: locks, constraints, exact money (25)	2	4	5	5	5
Delivery speed in 4–5 h: built-in auth, migrations, filters, files, admin (25)	3	3	2	3	5
Security and permission primitives (15)	2	4	5	3	4
Scalability path (15)	4	4	5	4	4
Maintainability and typing (10)	2	5	5	4	4
Testability of workflow and money rules (10)	3	4	5	4	4
Weighted score (out of 100)	**53**	**77**	**85**	**77**	**90**

4.3 Decision: FastAPI + SQLAlchemy 2 + PostgreSQL

> **On merit alone Django + DRF scores highest**, because it ships the most built-in features. **We choose FastAPI**, which ties with NestJS on merit, because the criteria it loses on are the cheap ones to close, while the risk it removes is the expensive one in a 4–5 hour build.

Reason	Detail
Equal on what is graded most	FastAPI scores 5/5 on transactional integrity, the same as Django and Spring Boot. Row locks, constraints and exact money come from PostgreSQL and SQLAlchemy, not from the web framework, so every guarantee in this document is unchanged.
Lowest delivery risk for this team	The team already builds and runs FastAPI + SQLAlchemy + PostgreSQL in production and has not used Django. In a 4–5 hour window a known toolset beats a larger feature list, and every line of the code must be explained in review.
Explicit code for a workflow that will be reviewed	No hidden framework behaviour: the transaction, the row lock, the permission check and the state machine are plain Python a reviewer can follow from router to database.
Typed contracts and free API documentation	Pydantic models validate every input and produce the OpenAPI specification and the `/docs` page automatically; the React client can generate its types from the same spec.

Reason	Detail
The gap is small and known	Django's lead comes from built-ins such as the user model, admin and filters. Table 4.4 lists each one and what replaces it. In total it is roughly an hour of known work.
What we accept	No free admin back-office (Swagger UI at `/docs` plus the seed script cover the demo), and a little more hand-written plumbing.

Other options

Option	Verdict and reason
Django + DRF	Highest merit score: user model, migrations, `select_for_update`, object permissions, filters, pagination, storage abstraction and an admin come built in. Not chosen because the team has no Django experience, so its speed advantage would be spent learning it, and its implicit conventions are harder to defend line by line.
Spring Boot	The strongest long-term enterprise fit: `@Transactional`, pessimistic locking, `BigDecimal`, Spring Security. It loses on delivery speed: entities, DTOs, repositories and security configuration roughly double the code for a 5-hour window. If the organisation runs on Java, this design maps to it one-to-one.
NestJS + PostgreSQL	Good structure and TypeScript end to end. Row locks need raw SQL in Prisma, and money needs a decimal library on every calculation because JavaScript numbers are floating point.
MERN	Not suitable. The "M" is the problem: the budget rule spans projects and requests, which needs one locked row and CHECK / foreign-key constraints that MongoDB does not have. Multi-document transactions need a replica set, and money would need `Decimal128` everywhere. The "ERN" part is fine, which is why the frontend still uses React.

4.4 What Django would have given us, and how the FastAPI MVP covers it

Built into Django	FastAPI MVP equivalent	Extra effort
User model, password hashing	`users` table + `bcrypt`	small
Migrations	Alembic with autogenerate from SQLAlchemy models	none
`transaction.atomic`, `select_for_update`	`with session.begin():` and `select(...).with_for_update()`	none
Model constraints	SQLAlchemy `CheckConstraint` / `UniqueConstraint`	none
Permission classes	Policy functions injected as FastAPI dependencies, e.g. `Depends(require("request.approve"))`	small, and more explicit
Filtering and pagination	Typed query parameters plus one shared `paginate()` helper	small
Storage abstraction	A `Storage` interface with `LocalStorage` now and `S3Storage` later	small
Throttling	`slowapi` rate limit on `/auth/login`	small
Admin back-office	Not replaced; Swagger UI at `/docs` is the fallback demo	accepted gap

4.5 Frontend, database and supporting choices

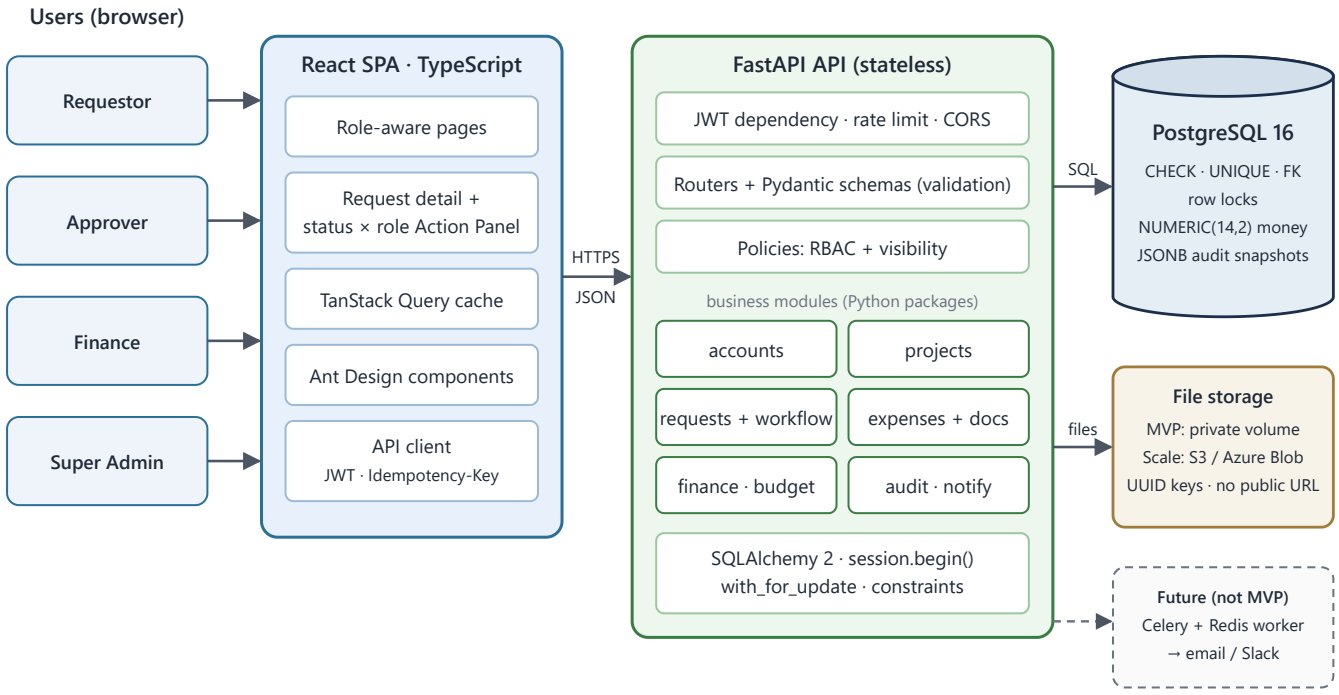
Choice	Alternatives considered	Why this one
SQLAlchemy 2 + Alembic	SQLModel, Tortoise ORM, raw SQL	Mature, explicit API for row locks (`with_for_update`), conditional updates and constraints; Alembic generates migrations from the models.

Choice	Alternatives considered	Why this one
Synchronous endpoints	async SQLAlchemy	Every money action is a short, locked transaction, and sync code is simpler to reason about under locks. FastAPI runs sync endpoints in a thread pool; individual endpoints can move to async later if needed.
Pydantic v2	Manual validation	Typed request and response models; `Decimal` fields with two places for money; the source of the OpenAPI spec.
React + TypeScript + Vite	Angular, Vue, server-rendered templates + HTMX	Largest ecosystem and hiring pool. A single-page app talks to the same REST API that a mobile app or integration could use later. HTMX would be quicker for plain forms but awkward for the per-item review panels.
Ant Design	Material UI, Chakra, hand-written CSS	Built for data-heavy business apps: tables with filters, forms, upload, step tracker and timeline ship ready-made, which is exactly this app's UI.
TanStack Query	Redux, plain fetch	Handles loading, error, caching and refetch-after-action, which the PDF lists under usability, without extra state code.
PostgreSQL 16	MySQL, SQLite, MongoDB	Row locks, enforced CHECK constraints, partial unique indexes, JSONB for audit snapshots, and row-level security for multi-tenancy later. SQLite cannot lock a single row, which the concurrency rules rely on.
JWT (PyJWT) + bcrypt	Server sessions	No session state on the server, so the API can run on many instances behind a load balancer. The user is reloaded on each call, so deactivation applies immediately.
Docker Compose	Manual install	Setup instructions are a deliverable; reviewers start database, API and web with one command.
pytest + httpx TestClient	unittest	Fixtures make role-by-role scenarios short; concurrency tests open separate sessions in parallel threads against real PostgreSQL.

5. System architecture

One deployable backend (a "modular monolith") serves a single-page frontend. The backend is split into six Python modules that match the business areas, so any of them can later become its own service without rewriting the others.

Figure 1 — System architecture (MVP)



The browser never talks to the database or the file store directly. Every file is downloaded through an API endpoint that checks the user may see the request it belongs to.

5.1 Business modules

Module (package)	Owns	Key responsibilities
accounts	users, invitations	login, invitation links, roles, activate / deactivate, `can_finance_ops` grant
projects	projects, project_members	project CRUD, activate / deactivate, budget changes (never below commitments), requestor assignment
fund_requests	fund_requests, workflow transition table	stages 1–2, resubmission, withdrawal, approver reassignment, the state machine itself
expenses	expense_items, documents	stage 4 submission, item review (stage 5), item verification (stage 6), upload validation
finance	disbursements, liquidations, budget_ledger	stage 3 payout, stage 6 liquidation, the budget engine (only code that changes project totals)
audit	audit_log, notifications	record every transition and money movement; in-app notifications

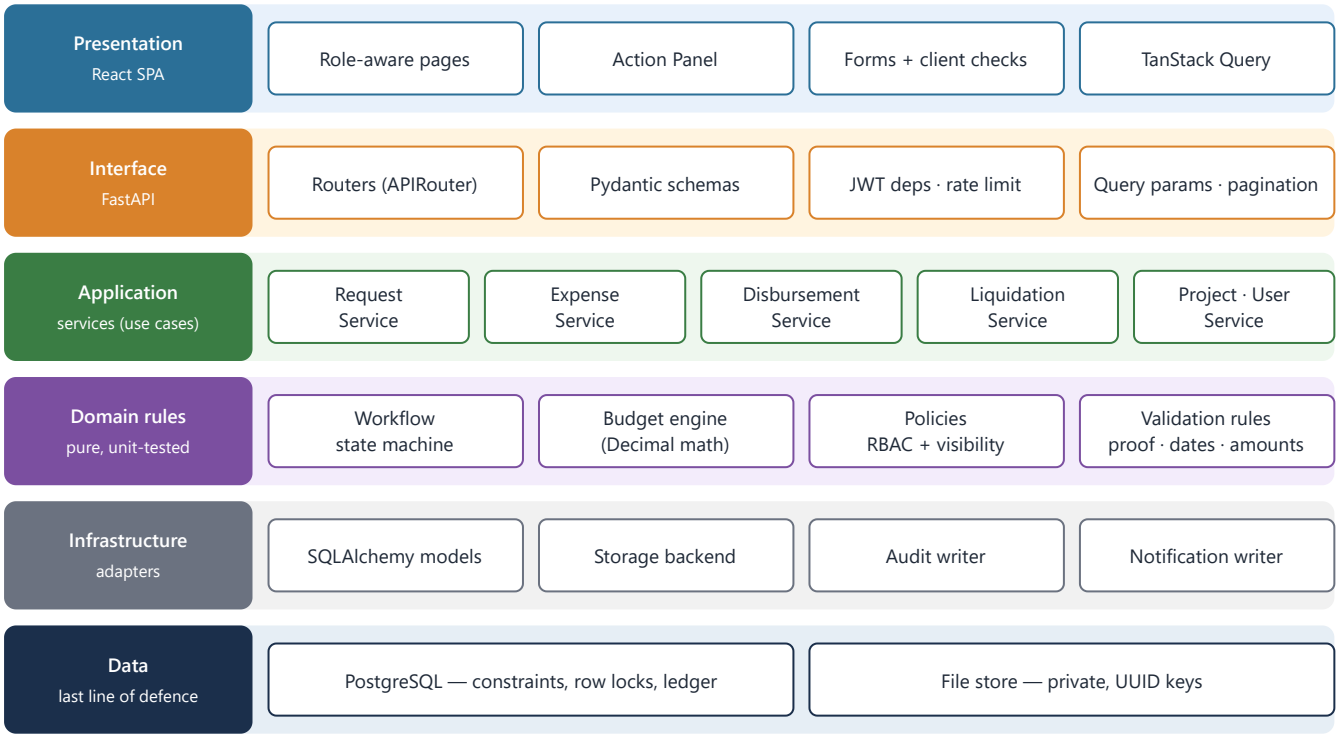
5.2 Why one deployable app and not microservices

Concern	Modular monolith (chosen)	Microservices
Budget reservation across project and request	One database transaction with one row lock	Needs distributed transactions or sagas
Build time in 4–5 h	One codebase, one deployment	Several services, network contracts, service discovery
Future split	Modules already follow business boundaries; one can be extracted when its load or team grows	Paid up front, before it is needed

6. Layered architecture

Each request travels down through the same layers. A layer may only call the layer directly below it. Business rules live in the domain layer, so they can be unit-tested without HTTP or a browser, and the database adds constraints as a final safety net.

Figure 2 — Layers and what lives in each



Dependencies point downward only. The domain layer imports nothing from FastAPI, which is what makes the state machine and budget maths testable on their own.

Layer	Owns	Must not
Presentation	Rendering, client-side checks for a better experience, showing only the actions the user may take	Be trusted for permissions or money maths; the backend repeats every check
Interface	HTTP, authentication, input parsing, mapping errors to 400 / 401 / 403 / 404 / 409 / 422, pagination	Contain business rules
Application	One method per use case; opens the database transaction; calls domain rules; writes audit and notification	Know about HTTP or the browser
Domain	Transition table, budget equations, permission and visibility rules, validation rules	Read files or speak HTTP
Infrastructure	ORM models, storage, audit and notification writers	Decide business outcomes

Layer	Owns	Must not
Data	CHECK / UNIQUE / FK constraints, row locks	—

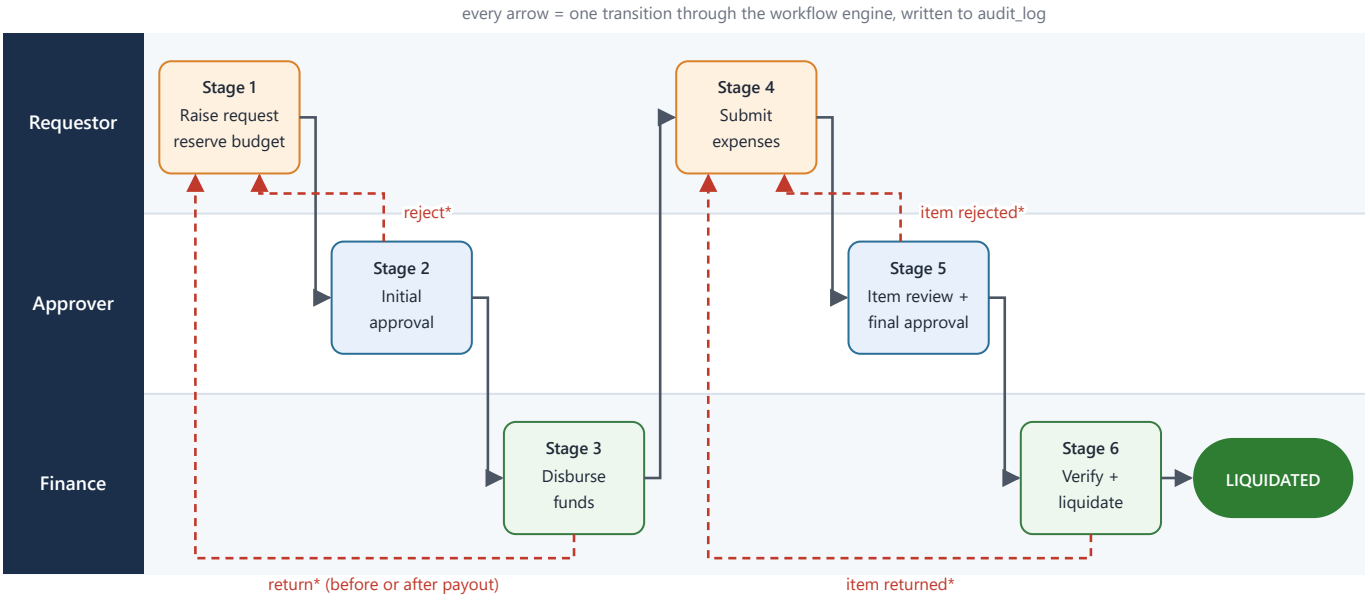
6.1 Code layout

```
fund-request/
├─ docker-compose.yml            db · api · web
├─ backend/
│  ├─ alembic/                   migrations (autogenerated from the models, reviewed by hand)
│  ├─ app/
│  │  ├─ main.py                 FastAPI app, routers, CORS, exception handlers (400/409/422)
│  │  ├─ core/                   config.py · db.py (session per request) · security.py (JWT, bcrypt)
│  │  │                          money.py (Decimal helpers) · errors.py · pagination.py · storage.py
│  │  ├─ modules/                each module: models.py · schemas.py · router.py · service.py · policies.py
│  │  │  ├─ accounts/            User, Invitation · invite / accept / users API
│  │  │  ├─ projects/            Project, ProjectMember
│  │  │  ├─ fund_requests/       FundRequest · workflow.py (TRANSITIONS)
│  │  │  ├─ expenses/            ExpenseItem, Document · validators.py (file type / size)
│  │  │  ├─ finance/             Disbursement, Liquidation, BudgetLedger · budget.py (budget engine)
│  │  │  └─ audit/               AuditLog, Notification · record()
│  │  └─ seed.py                 python -m app.seed: users, projects, sample requests at each stage
│  └─ tests/                     test_workflow.py · test_budget.py · test_concurrency.py · test_api_*.py
└─ frontend/
   └─ src/  api/ · auth/ · pages/ (Dashboard, Requests, RequestDetail, Projects, Users, Audit)
            components/ActionPanel/ (one panel per status × role) · components/ExpenseEditor/
```

7. Workflow and state machine

7.1 Six stages across the three working roles

Figure 3 — Six-stage flow by role (forward path solid, returns dashed red)



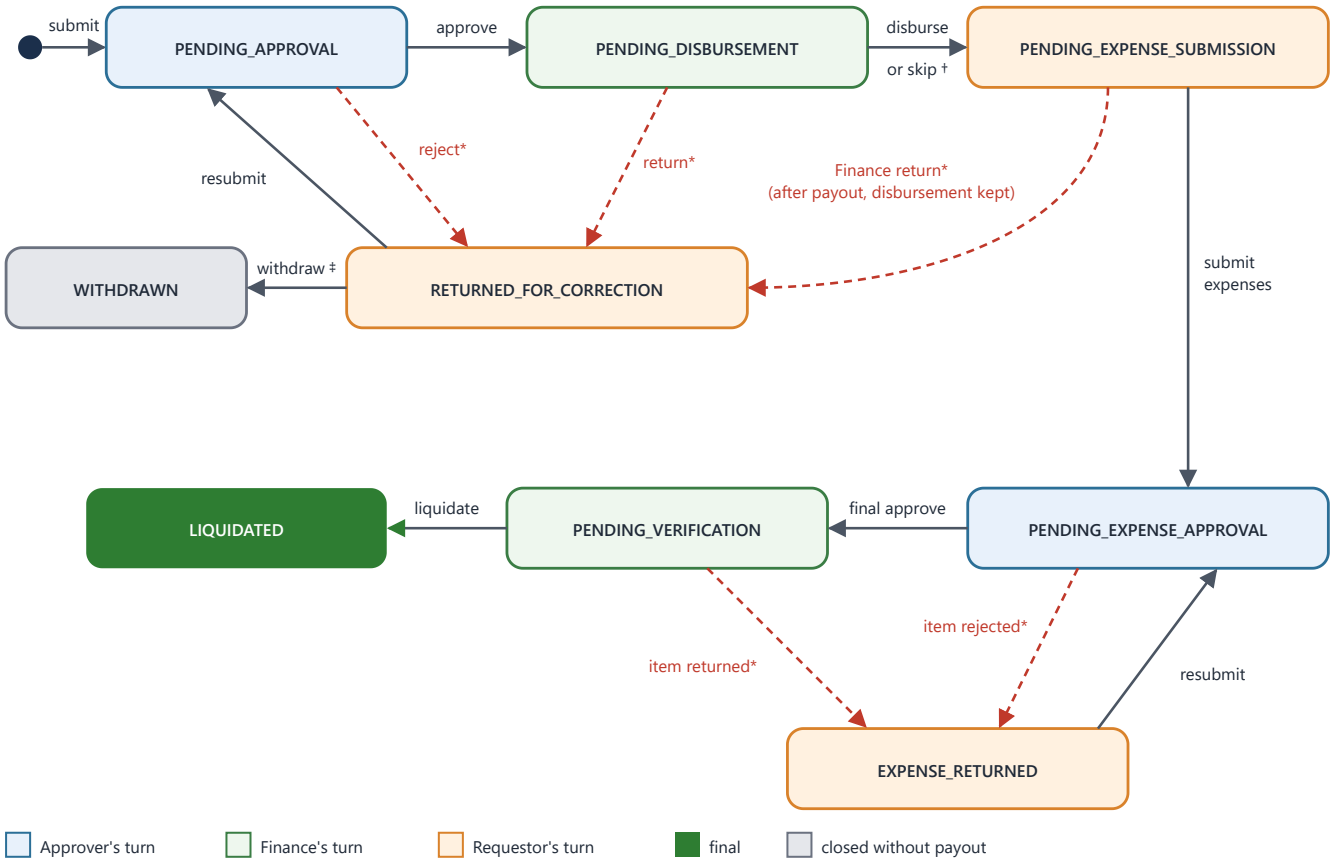
* A reason is mandatory on every rejection or return. Super Admin uses the Finance lane only when granted `can_finance_ops`. Returns to Stage 4 also send the corrected items back through Stage 5 before Finance sees them again.

Stage	Actor	What happens	Status after success	On rejection or failure
1 Raise	Requestor	Pick an eligible project and an approver, enter amount and purpose; budget reserved	PENDING_APPROVAL	Validation failure: stays at Stage 1, nothing saved
2 Approve	Assigned approver	Approve or reject the request	PENDING_ DISBURSEMENT	Rejected → Stage 1 (RETURNED_FOR_ CORRECTION), reservation released
3 Disburse	Finance	Record amount, date, transaction reference, notes; requestor notified	PENDING_EXPENSE_ SUBMISSION	Returned → Stage 1; any payout already made is kept
4 Expenses	Requestor	Add line items with proofs; total calculated; justification if above the request	PENDING_EXPENSE_ APPROVAL	Validation failure: stays at Stage 4
5 Review	Assigned approver	Approve or reject each item; final approval once all are approved	PENDING_VERIFICATION	Any item rejected → Stage 4 (EXPENSE_RETURNED)
6 Liquidate	Finance	Verify each item; liquidate; balances updated; requestor notified	LIQUIDATED	Any item returned → Stage 4, then Stage 5 again

Every rejection or return records the reason, the user, the time, the previous status and the new status. A resubmission always goes through every stage after the one it returns to; none can be skipped.

7.2 Request state machine

Figure 4 — Request statuses and allowed transitions (colour = whose turn it is)



* Reason mandatory. † Skipped automatically when the approved amount has already been paid out (request returned after payout). ‡ Only while nothing has been disbursed. Approver reassignment keeps the status at PENDING_APPROVAL and is allowed only there.

7.3 Transition table (the single source of truth)

This table is the `TRANSITIONS` dictionary in `app/modules/fund_requests/workflow.py`. The application layer looks up `(current status, action)`; if there is no row, the call is refused with HTTP 409.

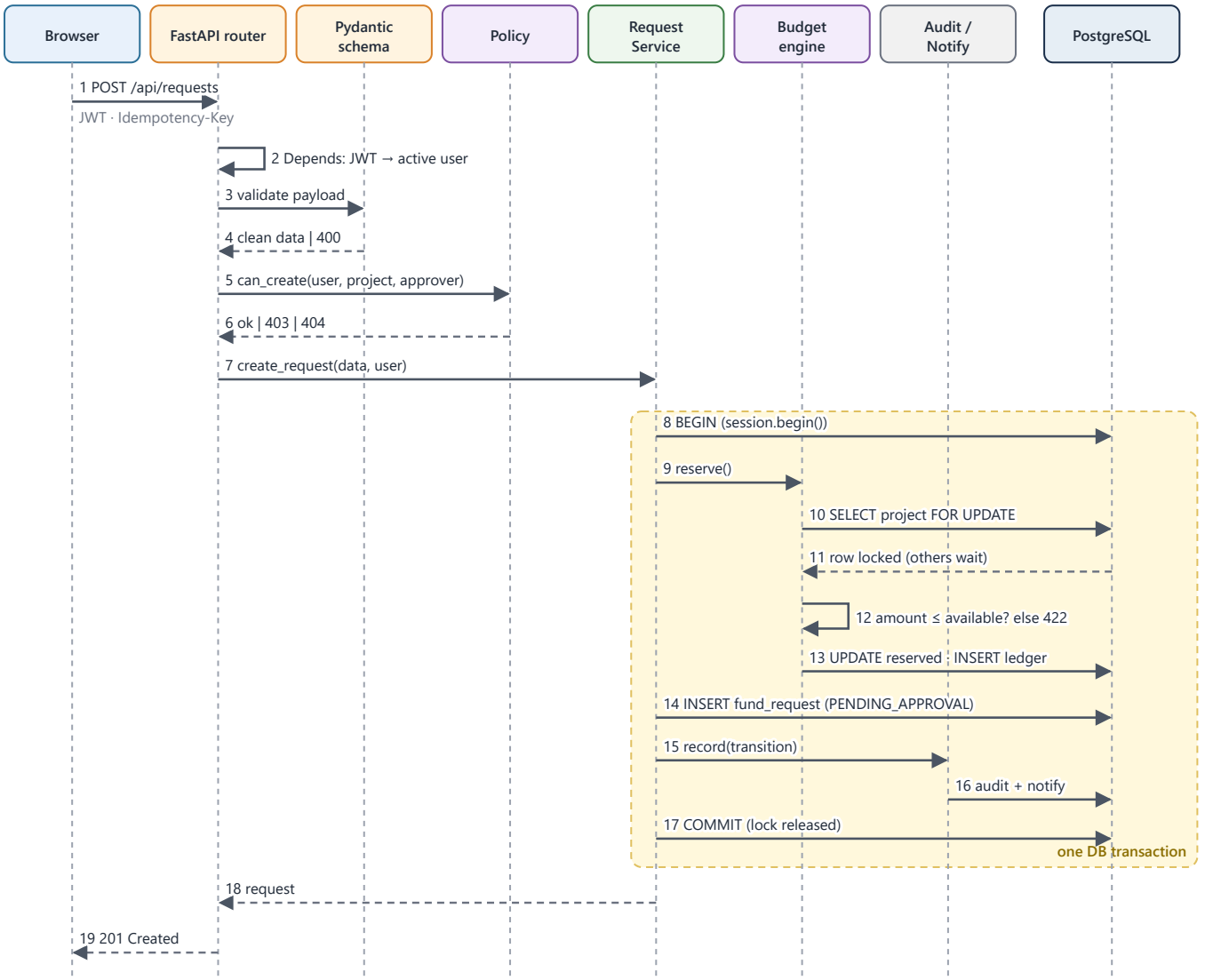
From	Action	Who	To	Guards and side effects
(new)	submit	Requestor	PENDING_APPROVAL	Project active and assigned to the requestor; approver has the Approver role and is not the requestor; lock project; amount ≤ available; RESERVE amount
PENDING_APPROVAL	approve	Assigned approver	PENDING_ DISBURSEMENT	amount_approved = amount_requested
PENDING_APPROVAL	reject (reason)	Assigned approver	RETURNED_FOR_ CORRECTION	RELEASE the request's reservation
PENDING_APPROVAL	reassign approver	Finance / Super Admin	PENDING_APPROVAL	New approver valid and not the requestor; audit old and new approver; notify both
RETURNED_FOR_ CORRECTION	resubmit	Requestor	PENDING_APPROVAL	Project still active; lock project; RESERVE max(new amount − already disbursed, 0)

From	Action	Who	To	Guards and side effects
RETURNED_FOR_ CORRECTION	withdraw	Requestor	WITHDRAWN	Only if nothing has been disbursed; RELEASE any reservation
PENDING_ DISBURSEMENT	disburse	Finance	PENDING_EXPENSE_ SUBMISSION	Amount = approved − already disbursed; idempotency key; reserved → outstanding; insert disbursement
PENDING_ DISBURSEMENT	*auto-skip*	System	PENDING_EXPENSE_ SUBMISSION	When approved − already disbursed ≤ 0; no new disbursement row
PENDING_ DISBURSEMENT	return (reason)	Finance	RETURNED_FOR_ CORRECTION	RELEASE the reservation
PENDING_EXPENSE_ SUBMISSION	return (reason)	Finance	RETURNED_FOR_ CORRECTION	Payout already made: disbursement row kept, money stays outstanding (see D3)
PENDING_EXPENSE_ SUBMISSION, EXPENSE_ RETURNED	submit expenses	Requestor	PENDING_EXPENSE_ APPROVAL	≥ 1 item; proof on each item; justification if total > requested; lock project and RESERVE / RELEASE the change in overage above the disbursed amount
PENDING_EXPENSE_ APPROVAL	review item	Assigned approver	*(unchanged)*	Item approved, or rejected with reason; approved items from earlier rounds stay approved
PENDING_EXPENSE_ APPROVAL	send back	Assigned approver	EXPENSE_RETURNED	All items reviewed and at least one rejected
PENDING_EXPENSE_ APPROVAL	final approve	Assigned approver	PENDING_ VERIFICATION	Every item approved
PENDING_VERIFICATION	verify item	Finance	*(unchanged)*	Item verified, or returned with reason
PENDING_VERIFICATION	send back	Finance	EXPENSE_RETURNED	Returned items go back to approval PENDING, so they pass Stage 5 again
PENDING_VERIFICATION	liquidate	Finance	LIQUIDATED	Every item verified; balance-return reference if underspent; lock project; settle (section 9); insert liquidation (unique per request)

8. How a request crosses the layers

The example below is Stage 1, raising a fund request, because it touches every layer and the concurrency rule. Every other action follows the same path; the table after the diagram lists what changes for each.

Figure 5 — "Raise fund request" end to end



If any step inside the yellow box fails, the whole transaction rolls back: no reservation, no request row, no audit row. A second request for the same project waits at step 10 until step 17 commits, then sees the reduced balance.

8.1 Layer path for every action

Action	Endpoint	Policy check	Service → domain	Locks	Writes
Raise / resubmit	`POST /requests` `PUT /requests/{id}`	Requestor; member of active project; approver valid and not self	RequestService → Budget.reserve	project, request	request, ledger RESERVE, audit, notify approver
Approve / reject	`POST …/approve | reject`	Assigned approver	RequestService → Workflow (+ Budget.release)	request (+ project)	status, ledger RELEASE, audit, notify requestor
Reassign approver	`POST …/reassign`	Finance / Super Admin; status PENDING_ APPROVAL	RequestService	request	approver_id, audit, notify both approvers

Action	Endpoint	Policy check	Service → domain	Locks	Writes
Disburse / return	`POST …/disburse | return`	Finance (or granted Super Admin)	DisbursementService → Budget.disburse	project, request	disbursement, ledger DISBURSE, audit, notify
Submit expenses	`POST …/expenses/submit`	Owning requestor; Stage 4 status	ExpenseService → Budget.adjust_overage	project, request	items, documents, ledger delta, audit, notify approver
Review item	`POST /expense-items/{id}/review`	Assigned approver; Stage 5	ExpenseService	request	item approval fields, audit
Final approve / send back	`POST …/final-approve | send-back`	Assigned approver	Workflow guard: all reviewed	request	status, audit, notify
Verify item	`POST /expense-items/{id}/verify`	Finance; Stage 6	ExpenseService	request	item verification fields, audit
Liquidate	`POST …/liquidate`	Finance; all items verified	LiquidationService → Budget.settle	project, request	liquidation (unique), ledger LIQUIDATE + RETURN / TOPUP, audit, notify
Download proof	`GET /documents/{id}`	Can see the parent request	Storage.open	—	access logged

Locks are always taken in the same order, project row first and request row second, so two transactions can never wait on each other in a circle (deadlock).

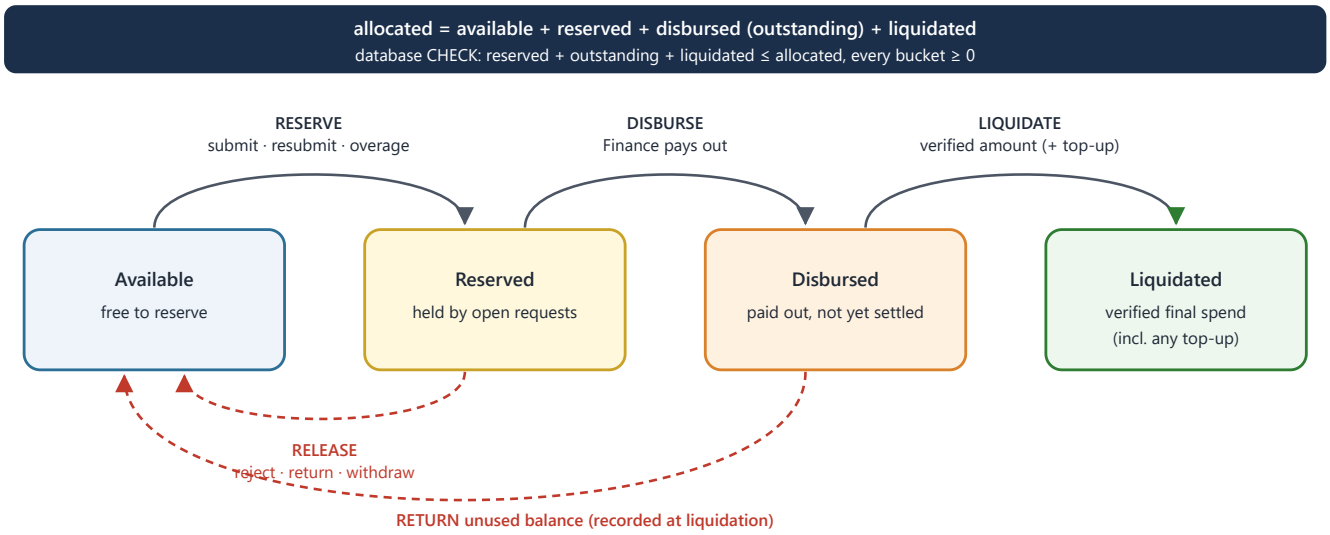
8.2 Error contract

Code	Meaning	Example
400	Input is malformed	Amount with three decimals, missing purpose, file over 5 MB. FastAPI's default 422 for schema errors is remapped to 400 by an exception handler, so 422 always means a business rule.
401	Not logged in, or token expired / user deactivated	Deactivated user tries to use an old token
403	Can see the request but may not do this	Requestor tries to approve their own request
404	Cannot see it (or it does not exist)	Requestor guesses another person's request ID; existence is not revealed
409	The request is not in the state this action needs	Second click on "Liquidate"; stale screen; action not in the transition table
422	A business rule says no	Amount above the project's available budget; final approval with a rejected item

9. Budget and money model

Every project's allocation is always split into four buckets. Money only moves between buckets through the budget engine, and every move writes one row to the append-only `budget_ledger`. The running totals on the project row make the dashboard fast; the ledger lets us recompute and check them at any time.

Figure 6 — Where project money sits and what moves it



Overspend above the disbursed amount is reserved when expenses are submitted (so it cannot be promised to someone else), and is paid as a top-up when the request is liquidated.

9.1 What each event does

Event	Bucket change	Ledger row(s)
Submit / resubmit request	reserved += max(amount − already disbursed, 0)	RESERVE
Reject, return to Stage 1, withdraw	reserved −= this request's reservation	RELEASE
Disburse X	reserved −= X; outstanding += X	DISBURSE
Submit expenses with total T	overage = max(T − disbursed, 0); reserve or release the change since the last submission	RESERVE / RELEASE
Liquidate with verified V, disbursed D	outstanding −= D; reserved −= overage reservation; liquidated += V	LIQUIDATE, plus RETURN (D − V) or TOPUP (V − D)
Change project budget	allocated = new value, refused if below reserved + outstanding + liquidated	ALLOCATE

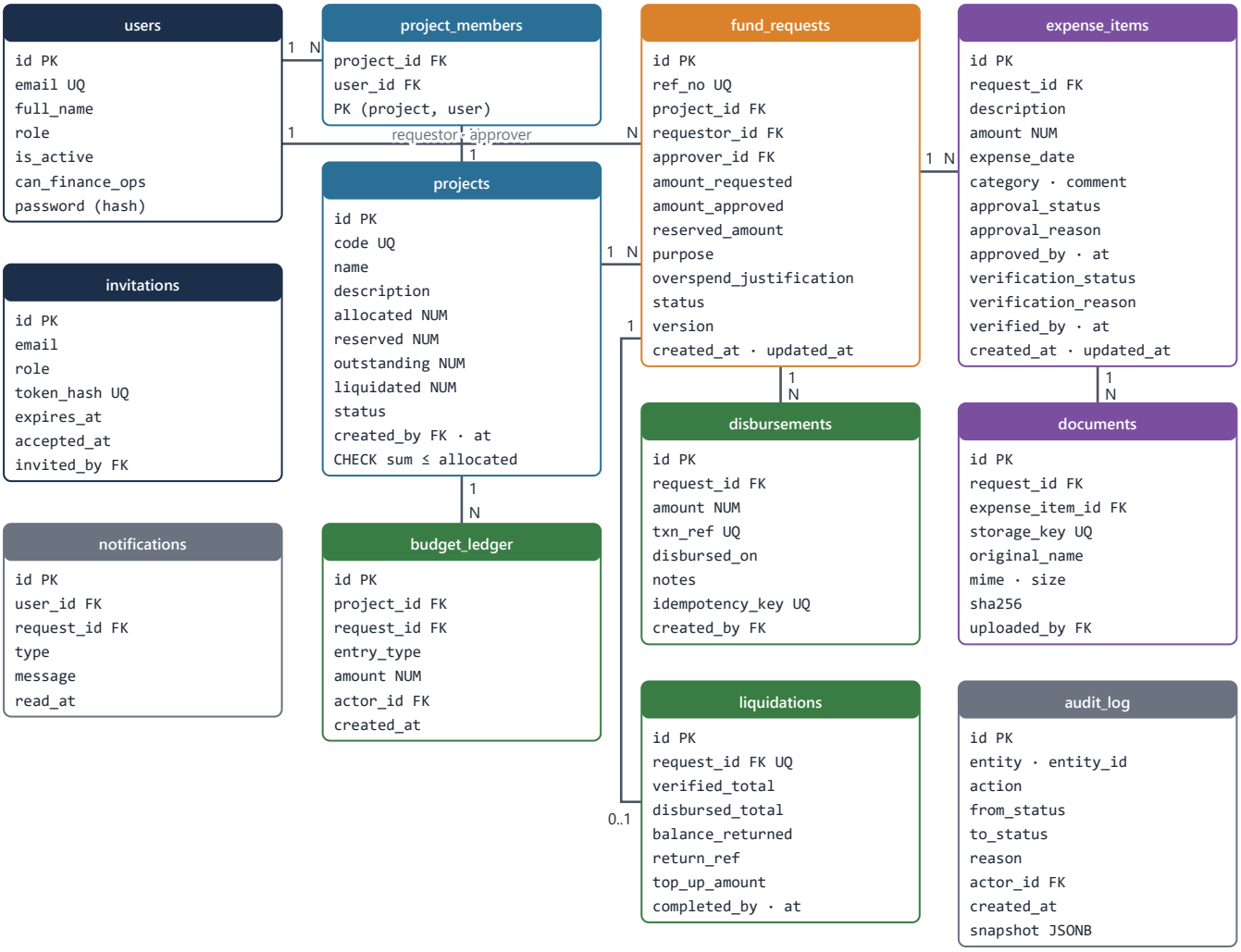
9.2 Worked example (project allocation 100,000)

Step	Reserved	Disbursed	Liquidated	Available
Start	0	0	0	100,000
R1 raised for 30,000	30,000	0	0	70,000
R1 approved and disbursed 30,000	0	30,000	0	70,000
R1 expenses 32,000 submitted with justification (overage 2,000 reserved)	2,000	30,000	0	68,000
R1 approved, verified 32,000, liquidated (top-up 2,000 paid)	0	0	32,000	68,000
Alternative: verified only 27,500 (balance return 2,500 recorded)	0	0	27,500	72,500

Every request also keeps its own figures, as the PDF requires: requested, approved, disbursed (sum of its disbursements), submitted expenses, verified expenses and liquidated amount.

10. Data model

Figure 7 — Entity relationship diagram



NUM = NUMERIC(14,2). Every `*_by`, `actor_id` and `user_id` column references `users`; those lines are left out to keep the diagram readable. Header colour matches the owning module.

10.1 Integrity guarantees enforced by the database

The application checks every rule first and returns a friendly error. The database repeats the most important rules so that a bug, a race or a manual script still cannot corrupt the money.

Constraint	What it prevents
`CHECK reserved + outstanding + liquidated ≤ allocated`, every bucket ≥ 0	A project committing more money than it has
`UNIQUE liquidations.request_id`	Liquidating the same request twice
`UNIQUE disbursements.idempotency_key`, `UNIQUE txn_ref`	Recording the same payout twice; reusing a bank reference
`CHECK amount > 0` on every money column	Zero or negative requests, payouts and expenses
`CHECK requestor_id ≠ approver_id`	Self-approval, even through a direct database write
`PRIMARY KEY (project_id, user_id)` on project_members	Duplicate project assignments

Constraint	What it prevents
`UNIQUE` projects.code, users.email, fund_requests.ref_no	Duplicate identifiers
Foreign keys with `ON DELETE PROTECT`	Deleting history; records are deactivated instead
`version` column, checked on every status update	Lost updates from an out-of-date screen
Indexes on `(status, project_id)`, `requestor_id`, `approver_id`, `created_at`	Slow dashboards and filters as data grows

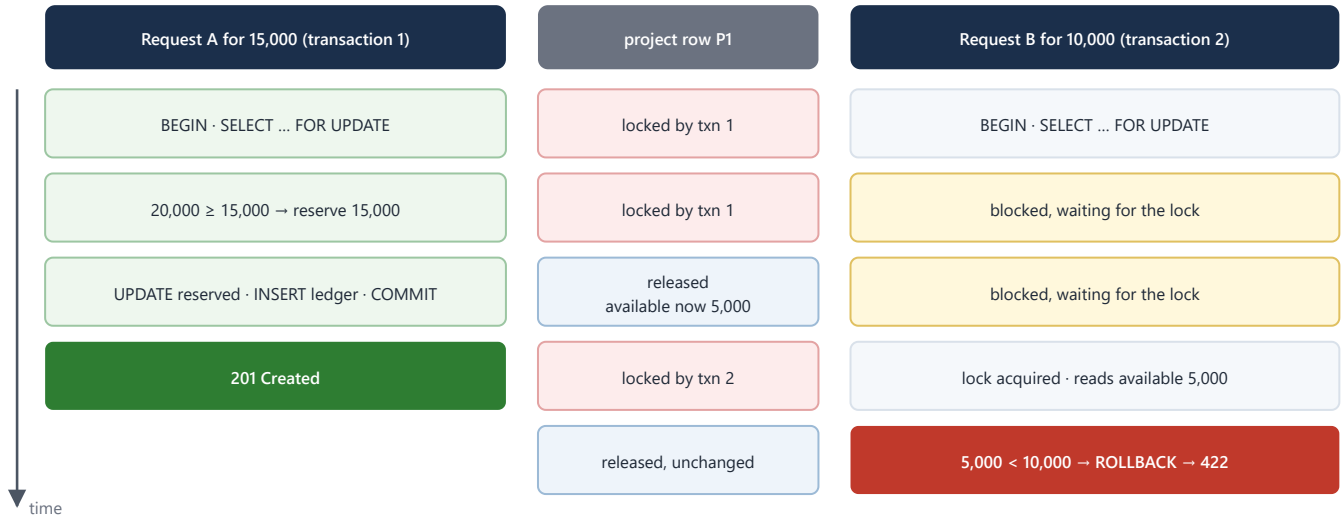
10.2 How history is preserved

- Requests and items are never deleted; status fields change, and every change writes an `audit_log` row with a JSON snapshot of the record before the change.
- Rejected expense items are corrected in place, but the snapshot keeps what the approver rejected and why.
- Disbursements and liquidations are separate tables, and the ledger is only ever appended to.

11. Concurrency and edge cases

11.1 Two requests racing for the last of a budget

Figure 8 — Row lock serialises two reservations on the same project (available 20,000)



Without the lock both transactions would read 20,000 and both would succeed, committing 25,000 against a 20,000 balance. The CHECK constraint would still reject the second commit, but the lock gives a clean 422 message instead of a database error.

11.2 Concurrency and duplicate actions

#	Case	Handling
1	Two requests reserve the same budget at once	Project row locked with `with_for_update()`; balance checked inside the lock; CHECK constraint as backstop (Figure 8)
2	Double click or network retry on approve, disburse or liquidate	Status update is conditional: `UPDATE … WHERE id = :id AND status = :expected AND version = :v`. Zero rows updated → 409. The button is also disabled while the call is running.
3	Two Finance users disburse the same request	Request row locked and status checked; `Idempotency-Key` header stored with a UNIQUE constraint; total disbursed may not exceed the approved amount
4	Two Finance users liquidate at the same time	Status check plus `UNIQUE liquidations.request_id`; the second call gets 409
5	Approver reassigned while the old approver is approving	Both lock the request row; reassignment only allowed in PENDING_APPROVAL; whichever commits first wins, the other gets 409
6	Action taken from an out-of-date screen	Client sends the `version` it loaded; mismatch → 409 "This request changed, please reload"
7	Deadlock between two money operations	Locks always taken in the same order: project, then request
8	Failure halfway through	Status, ledger, audit and notification are written in one transaction. Files are saved first and deleted if the transaction rolls back.

11.3 Workflow and money edge cases

#	Case	Handling
9	Request returned to Stage 1 after funds were paid out	Disbursement row kept and still counted as outstanding. On resubmit only the shortfall is reserved; after re-approval only the shortfall can be disbursed; if there is none the request skips to Stage 4. Any excess is settled at liquidation as a balance return.

#	Case	Handling
10	Expenses exceed the requested amount	Justification required. The approver's item approvals are the extra approval. Overage above the disbursed amount must fit the available budget and is reserved.
11	Expenses below the disbursed amount	Finance must enter the balance-return reference at liquidation; the unused money goes back to available.
12	Expense report resubmitted after rejection	Approved items are locked; rejected items editable; new items allowed. Items returned by Finance go back to approval PENDING so they pass Stage 5 again.
13	Final approval with an unreviewed or rejected item	422 from the backend; the button is hidden in the UI
14	Budget reduced below commitments	422; allocation cannot go below reserved + outstanding + liquidated. Allowed changes write an ALLOCATE ledger row and an audit row.
15	Project deactivated with requests in progress	New requests and Stage 1 resubmissions blocked; requests past Stage 1 continue to liquidation; history stays visible
16	Requestor removed from a project	Existing requests stay visible (visibility is based on involvement); no new requests on that project
17	Approver deactivated mid-flow	Before approval: Finance reassigns. After approval the PDF ties expense review to the same approver, so the MVP blocks deactivation of an approver with open Stage 5 work and lists those requests.
18	Self-approval	Rejected by the policy and by a database CHECK; one role per user removes the "two roles" loophole
19	Bad amounts or dates	Amount > 0, two decimals, upper limit; expense date not in the future; at least one expense item
20	Withdrawing after payout	Not allowed; money that has left must be accounted for through liquidation

11.4 Security edge cases

#	Case	Handling
21	Guessing another request's ID	All queries go through a visibility filter; not visible → 404, so existence is not revealed
22	Opening someone else's receipt	Files only through `/documents/{id}`, which runs the parent request's visibility check; sent as a download
23	Malicious upload	PDF / JPG / PNG only, checked by file content, not just extension; 5 MB limit; random UUID file name; stored outside any public folder
24	Deactivated user with a valid token	User reloaded on every call and `is_active` checked; short access-token lifetime
25	Invitation link reused or leaked late	Token stored hashed, single use, expires after 72 hours
26	Password guessing	`slowapi` rate limit on the login endpoint; minimum password length checked in the schema

12. Security

Authentication

- Email and password; passwords hashed with bcrypt; minimum length enforced.
- Short-lived JWT access token and a refresh token; the user record is reloaded on every request.
- Accounts are created only by invitation; an invitation fixes the role.

Authorisation, in two steps

- **Can see?** A visibility filter applied to every query: Finance and Super Admin see all; requestors and approvers see requests where they are the requestor or the approver.
- **Can do?** A permission check per action (for example `request.approve`) mapped from the role, plus the object rule (for example "is the assigned approver"). Checks use permission names, not role names, so custom roles can be added later without touching the views.

Files

- Allowed types checked by reading the file's first bytes; size limit; SHA-256 stored.
- Stored under a random key in a private location; never served by the web server directly.
- Moving to S3 later: private bucket, short-lived signed URLs issued only after the same visibility check.

Transport and platform

- HTTPS in every non-local environment; CORS limited to the web origin.
- Secrets from environment variables, never in the repository.
- SQLAlchemy bound parameters against SQL injection; Pydantic validation on every endpoint.

13. API and screens

Area	Endpoints
Auth	`POST /auth/login` · `POST /auth/refresh` · `POST /auth/accept-invite` · `GET /me`
Users (Super Admin)	`GET /users` · `POST /users/invite` · `PATCH /users/{id}` (role, active, can_finance_ops)
Projects	`GET|POST /projects` · `PATCH /projects/{id}` · `PUT /projects/{id}/members` · `GET /projects/{id}/ledger`
Requests	`GET /requests?project&status&requestor&approver&from&to&page` · `POST /requests` · `GET|PUT /requests/{id}` `POST /requests/{id}/approve | reject | reassign | withdraw | disburse | return`
Expenses	`PUT /requests/{id}/expenses` · `POST /requests/{id}/expenses/submit` · `POST /expense-items/{id}/review | verify` `POST /requests/{id}/final-approve | send-back | liquidate`
Documents	`POST /documents` (multipart) · `GET /documents/{id}`
History	`GET /requests/{id}/history` · `GET /audit` · `GET /notifications` · `POST /notifications/{id}/read`

FastAPI generates the OpenAPI documentation automatically and serves it at `/docs`, which doubles as a backend demo if the UI is not finished.

13.1 Screens

Screen	Who	Purpose
Login · Accept invite	everyone	Sign in; set a password from an invitation link
Dashboard	everyone	"My pending actions" by status, counts, recent notifications
Requests	everyone (filtered)	List with filters for project, requestor, approver, status, date; pagination; new request
Request detail	involved users, Finance, Super Admin	Step tracker, money figures, expense items, documents, history timeline and the Action Panel below
Projects	Finance, Super Admin	Budget bars (available / reserved / disbursed / liquidated), edit, activate, members, ledger
Users	Super Admin	Invite, change role, activate / deactivate, grant finance operations
Audit log	Finance, Super Admin	Search all transitions and money movements

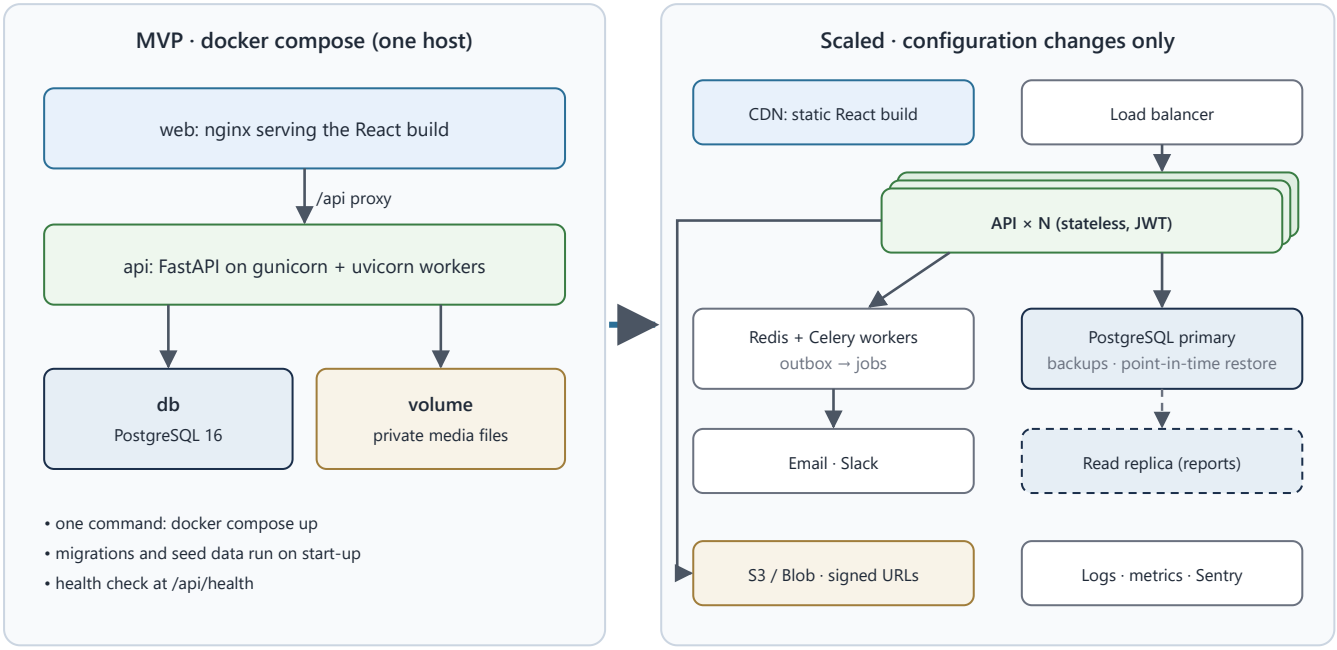
13.2 Action Panel: one component, driven by status and role

Instead of building a separate page per stage, the request detail page shows one panel whose content is chosen from the request's status and the viewer's role. This saves roughly six screens and mirrors the backend transition table, so the UI never offers an action the backend would refuse.

Status	Requestor	Approver	Finance (and granted Super Admin)
PENDING_APPROVAL	read-only	Approve · Reject (reason)	Reassign approver
RETURNED_FOR_CORRECTION	Edit and resubmit · Withdraw	read-only	read-only
PENDING_DISBURSEMENT	read-only	read-only	Disburse (amount, date, reference, notes) · Return (reason)
PENDING_EXPENSE_SUBMISSION	Expense editor · Submit	read-only	Return to Stage 1 (reason)
PENDING_EXPENSE_APPROVAL	read-only	Approve / reject each item · Final approve · Send back	read-only
EXPENSE_RETURNED	Fix rejected items · Resubmit	read-only	read-only
PENDING_VERIFICATION	read-only	read-only	Verify / return each item · Liquidate
LIQUIDATED · WITHDRAWN	summary	summary	summary and liquidation record

14. Scalability roadmap

Figure 9 — From one host to a scaled deployment, with the same code



The API keeps no state between calls, so more instances can be added behind a load balancer. Files, jobs and reporting move out to managed services through interfaces that already exist in the MVP.

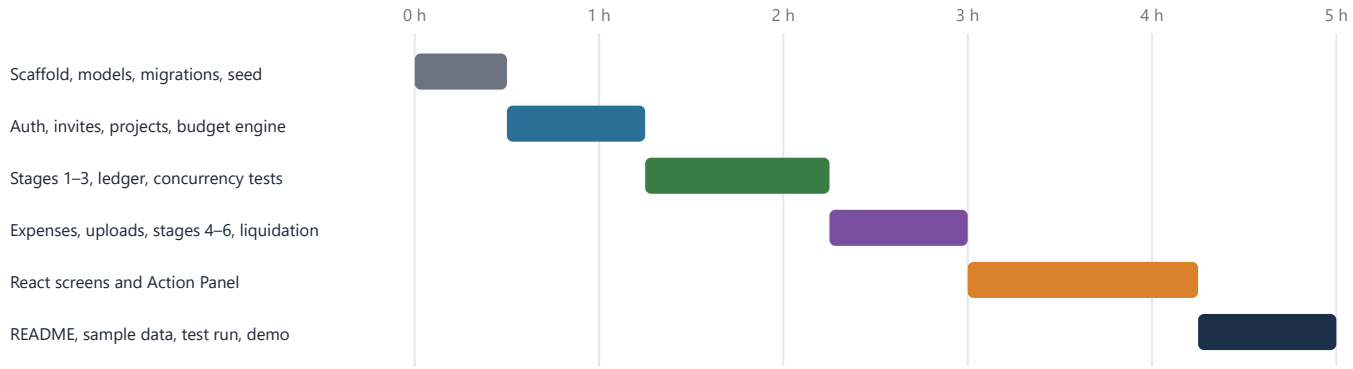
MVP choice	Scaling step	When it is needed
Stateless JWT API	Run N instances behind a load balancer; autoscale on CPU / latency	More concurrent users
`Storage` interface	Add an `S3Storage` class (boto3) selected by a setting; signed URLs; virus scan on upload	File volume, compliance
Notifications table	Add an outbox table written in the same transaction, and Celery workers that send email / Slack, so no message is lost	Email or chat notifications required
Transition table in code	Move transitions and approval rules into the database: multi-level approval, amount thresholds, per-department rules	Different approval policies
Permission names, not role names	Custom roles stored in the database and edited by Super Admin	More roles than the four we have
Append-only budget ledger	Nightly reconciliation job; exports to the accounting / ERP system	Finance audit, integrations
Indexes and pagination	Read replica for reports; monthly partitions for `audit_log`; caching for dashboard counts	Large data volumes
Modules by business area	Extract a module (for example documents or notifications) into its own service	Separate teams or very different load
Single organisation	Add `org_id` to every table and PostgreSQL row-level security	Serving several companies

15. Testing strategy

Level	What is tested	How
Workflow unit tests	Every (status, action, role) combination: allowed ones move to the right status, all others are refused	Table-driven pytest over `TRANSITIONS`; no HTTP
Budget unit tests	Every event in section 9.1, the worked example, overspend, underspend, return after payout	Pure functions with `Decimal` inputs
API integration tests	Full happy path; each rejection route from the PDF's section 5 table; role and visibility rules (403 vs 404)	httpx `TestClient` with one fixture user per role
Concurrency tests	Two parallel reservations on one project; double liquidation; double disbursement with the same key	Separate sessions in parallel threads against real PostgreSQL
Upload tests	Wrong type with a renamed extension, oversize file, download by a user who cannot see the request	TestClient with sample files
Manual demo script	One request through all six stages with one rejection at Stage 2 and one at Stage 5, using the seed users	Written in the README

16. Delivery plan (4–5 hours)

Figure 10 — Build order; backend logic first, because it is what is graded



> **Safety net:** after 3 hours the whole flow already works through the API and its Swagger UI at `/docs`, so the deliverables can be demonstrated even if the React screens need more time.

17. Design decisions log

The problem statement asks the implementer to define several rules. These are our answers, each with the reason behind it.

ID	Decision	Reason
D1	Budget is reserved when a request is submitted and released when it returns to Stage 1 or is withdrawn	FR-06 asks for reservation rules. Reserving at submit stops two pending requests promising the same money; releasing on return frees it for others while the request is being corrected.
D2	One approver per request; reassignment only while PENDING_APPROVAL, by Finance or Super Admin	Stated in the PDF (FR-17, section 2.2). The same approver reviews the expenses.
D3	Finance may return a request to Stage 1 after payout; the disbursement is kept, only a shortfall can be paid later, and a zero shortfall skips straight to Stage 4	The PDF requires this case to be handled explicitly and forbids a second payout for the same obligation.
D4	Spending above the request needs a justification; the approver's item approvals are the extra approval; spending above the payout must fit the project's available budget, is reserved at submission and paid as a top-up at liquidation	The PDF says a justification alone does not authorise overspending. This ties the extra money to an approval and to the budget check.
D5	Unused money is recorded as a balance return with a reference at liquidation and goes back to the project's available budget	The PDF asks how unused funds are returned and recorded.
D6	A proof document is required on every expense item (a setting can relax this later)	The PDF allows optional proof "if permitted by business rules"; requiring it is the safer default for an audit trail.
D7	One role per user; Super Admin gets money actions only through the `can_finance_ops` flag	Removes self-approval loopholes; matches "must be explicitly granted".
D8	Invitations are links with a single-use token, shown on screen in the MVP	No mail server is needed for the demo; email delivery plugs into the outbox later.
D9	Approved items stay locked on resubmission; items returned by Finance go back through approval	The PDF keeps approved items approved and forbids skipping a stage on resubmission.
D10	Withdrawal only from RETURNED_FOR_CORRECTION and only before any payout	Keeps abandoned requests from holding budget forever, without losing track of money already paid.
D11	An inactive project blocks new requests and Stage 1 resubmissions; requests past Stage 1 continue	The PDF asks to prevent new requests but preserve access to existing ones.
D12	A request the user cannot see returns 404, not 403	Does not reveal that the request exists.

> Each decision is repeated in the project README so that reviewers can find the business rules next to the setup instructions, as the deliverables require.